

DESIGN AND ANALYSIS OF ALGORITHM

WEEK 3

Q)1

Question 1 | Correct | Mark 1.00 out of 1.00 | Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m - Size of array

Next m lines Contains m numbers - Elements of an array

Output Format

First Line Contains Integer - Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int countZeros(int a[], int low, int high){
3     if(low==high)
4         return 0;
5
6     if(a[low]==0)
7         return high-low+1;
8     int mid=(low+high)/2;
9
10    return countZeros(a,mid+1,high);
11 }
12
13 int main(){
14     int n;
15     scanf("%d", &n);
16     int a[n];
17     for(int i=0;i<n;i++){
18         scanf("%d", &a[i]);
19     }
20     printf("%d", countZeros(a, 0, n-1));
21     return 0;
22 }
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓
✓	10 1 1 1 1 1 1 1 1 1	0	0	✓
✓	8 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1	2	2	✓

Q)2

Given an array **nums** of size **n**, return the majority element.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: **nums** = [3,3,3]

Output: 3

Example 2:

Input: **nums** = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5 \times 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

For example:

Input	Result
3 3 3 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int majorityElement(int nums[],int n){
3     int candidate=nums[0];
4     int count = 1;
5
6     for(int i=1;i<n;i++){
7         if(nums[i]==candidate)
8             count++;
9         else
10            count--;
11
12        if(count==0){
13            candidate=nums[i];
14            count=1;
15        }
16    }
17    return candidate;
18 }
19 int main(){
20     int n;
21     scanf("%d",&n);
22
23     int nums[n];
24     for(int i=0;i<n;i++){
25         scanf("%d",&nums[i]);
26     }
27     printf("%d",majorityElement(nums,n));
28
29     return 0;
30 }
```

	Input	Expected	Got	
✓	3 3 3 3	3	3	✓

Passed all tests! ✓

Q)3

Problem Statement:
Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format
First Line Contains Integer n - Size of array
Next n lines Contains n numbers - Elements of an array
Last Line Contains Integer x - Value for x

Output Format
First Line Contains Integer - Floor value for x

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int main(){
3     int n,x;
4     scanf("%d",&n);
5
6     int a[n];
7
8     for(int i=0;i<n;i++){
9         scanf("%d",&a[i]);
10    }
11    scanf("%d",&x);
12
13    int low=0,high=n-1;
14    int floor=-1;
15
16    while(low<=high){
17        int mid=(low+high)/2;
18        if(a[mid]<=x){
19            floor=a[mid];
20            low=mid+1;
21        }
22        else{
23            high=mid-1;
24        }
25    }
26    printf("%d",floor);
27    return 0;
28 }
```

	Input	Expected	Got	
✓	6	2	2	✓
	1			
	2			
	8			
	10			
	12			
	19			
	5			
✓	5	85	85	✓
	10			
	22			
	85			
	100			
	129			
	100			
✓	7	9	9	✓
	3			
	5			
	7			
	9			
	11			
	13			
	15			
	18			

Passed all tests! ✓

Q)4

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n - Size of array
Next n lines Contains n numbers - Elements of an array
Last Line Contains Integer x - Sum Value

Output Format

First Line Contains Integer - Element1
Second Line Contains Integer - Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int findPair(int a[],int left, int right,int x, int *e1,int *e2){
3     if(left==right)
4         return 0;
5     if(a[left]+a[right]==x){
6         *e1=a[left];
7         *e2=a[right];
8         return 1;
9     }
10    if(a[left]+a[right]<x)
11        return findPair(a,left+1,right,x,e1,e2);
12    return findPair(a,left,right-1,x,e1,e2);
13 }
14
15
16 int main(){
17     int n,x;
18     scanf("%d",&n);
19
20     int a[n];
21     for(int i=0;i<n;i++)
22         scanf("%d",&a[i]);
23     scanf("%d",&x);
24     int e1,e2;
25     if(findPair(a,0,n-1,x,&e1,&e2)){
26         printf("%d\n", e1);
27         printf("%d\n", e2);
28     }
29     else{
30         printf("No");
31     }
32     return 0;
33 }
```

	Input	Expected	Got	
✓	4 4 2 4 8 10 14	4 10	4 10	✓
✓	5 2 4 6 8 10 100	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Q)5

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no. of elements in the list n
The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```
1 #include <stdio.h>
2 void quickSort(int a[], int low, int high){
3     if(low<high){
4         int i=low;
5         int j=high;
6         int pivot=a[low];
7
8         while(i<j){
9             while(a[i]<=pivot && i<high)
10                 i++;
11             while(a[j]>pivot)
12                 j--;
13             if(i<j){
14                 int temp=a[i];
15                 a[i]=a[j];
16                 a[j]=temp;
17             }
18         }
19         a[low]=a[j];
20         a[j]=pivot;
21         quickSort(a,low,j-1);
22         quickSort(a,j+1,high);
23     }
24 }
25
26 int main(){
27     int n;
28     scanf("%d",&n);
29     int a[n];
30
31     for(int i=0;i<n;i++){
32         scanf("%d",&a[i]);
33         quickSort(a,0,n-1);
34     }
35     for(int i=0;i<n;i++){
36         printf("%d", a[i]);
37         if(i<n-1)
38             printf(" ");
39     }
40     return 0;
41 }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marked for this submission: 5 / 20 (1.00)